

13/11/25

Software Development Life Cycle SDLC

SDLC $\Rightarrow$ Roadmap for developing software
 $\Rightarrow$ define each stage
starting idea to final product maintenance
"design, develop and test"
Goal $\rightarrow$ high quality and maintainable software that meets user requirements.

Stages $\rightarrow$ all things should be discussed and finalized
Planning & Requirement Analysis

Defining Requirements What

Designing How

Building

Testing

Deployment & Maintenance

SRS $\Rightarrow$ Software Requirement Specifications

hosting or putting the project on internet

Stage 1: Planning and Requirement Analysis

(All available informations)

Planning $\rightarrow$ Define project scope $\rightarrow$ Set objectives and goals $\rightarrow$ Resource planning

Stage 2: Defining Requirements

Defining $\rightarrow$ Functional Requirements $\rightarrow$ Technical Requirements $\rightarrow$ Reviews & Approved.

requirements getting approval from users, market analysts and stakeholders

Stage 3: Defining Architecture

Evaluating all the possible factors

↳ most practical & logical design chosen

Design → low level design → high level design

DDS ⇒ design documentation

Stage 4: Developing product (Building)

developers use code as per design

languages like C/C++, python, java

Develop → Coding Standard → Scalable code → Version control → Code Review

Stage 5: Product Testing & Integration

Documentation → essential

Training → to improve current performance

System testing → Manual testing → Automated testing

Stage 6: Deployment & Maintenance

Based on the organization strategy.

Deployment & Maintenance → release → Deployment → automation
→ Maintenance planning

Maintenance → Feedback

Deployment → Making the project live for the users after testing

ex: Hosting a web app on AWS, render
Release → Users can now access system ex: visit & download

- functional requirements - what the system (actions or operation) should do, the specific function or feature it must perform.
- non-functional requirements - How the system (quality or constraints) should behave.

Feasibility of Software Development.

- * Check whether the project is possible and practical to build with available time, budget, technology and resources.

SRS → Software Requirement Specifications

- Detailed document that what the software do and how it should perform before coding.
- Bridge between client ^{wants software} & developer ^{build software} team.

Types

- i) Functional → what should do Ex: login, register, submit complaints
- ii) Non-functions → How behave Ex: Respond within 2s

Planning phase

⇒ to analyze and plan everything needed to build the software include

goal, scope, resource and time.
Ex: Building house where to build, how cost, what
Requirement analysis (avoid confusion) should do?

- Collecting and understanding what the client wants, developer know what to do and how to do.

Ex: Sitting with the client and what they want.

Designing phase

- planning how we built the software
- Evaluate all factors and chose the most practical thing is chosen

Design

↓

Low level Design [overall structure ex: modules, system architecture, data flow] user, admin, map, architecture

↓

High level Design [focus on each module room ex: writing pseudocode] like electrician & plumber.

Development

Implementation / coding phase:

- design into code
- setting environment, working software is built

Testing phase:

Check the quality, functionality and performance of software

Deployment phase

- the application is hosted on a server or cloud platform.
- software should be accessible to real users over Internet.

Maintenance phase

- After deployment, it should be monitored, to enhance the system performance, fix bugs and update it based on users feedback.

SDLS Models

* Different approaches or strategies to organize the seven phase

1. Waterfall Model (Traditional Development Method)

- One model must finish, another should execute.

⇒ Plan → Analysis → Design → Implement → Test → Deploy → Maintain

• Best for: small projects & goal is clearly defined

Dis: Hard to change

2. Iterative Model

- Build the software in small parts (Iterations)

• Each part should be improved based on feedback

monitoring → watch if any issue fix them quickly.

SDLC Models

⇒ Guide for planning, execution and delivery of software projects

- Waterfall model ⇒ linear & sequential approach
- Used in large, complex objects in IT
- Used in the place where goal is clear.
- Sequential flow

Testing

- Alpha testing - development team
- Beta testing - customers
- Acceptance testing - to determine whether accept the delivered software or not

Maintenance

- Preventive → change to prevent future problem correct after release.
 - Corrective maintenance - not discovered during the development phase
 - Perfective maintenance - Enhance system functionality
 - Adaptive maintenance - porting the software works in new environment
- Stakeholders - individual or groups who have an interest involved directly or indirectly involved in projects.

Ex: Clients, developers, users.

Used for: Medium projects, requirements involve
Drawback: Require good planning & version control

3. Spiral Model (bridge)

* Combines Iterative + Risk Analysis

planning → ^{risk} Analysis → Design → Implementation → Evaluation

Used for: large, complex, high-risk projects

Drawback: Expensive & time consuming

4. V-Model (Verification & Validation)

• Each development phase has corresponding test phase

Best for: Safety critical projects (safety test)

Drawback: Changes are difficult

5. Agile Model (Most popular) (teamwork)

• flexible fast, customer focused.

• Sprint → fixed, short cycle (1-4 weeks)

and deliver usable software.

Used for: Projects with changing requirements,
startups, dynamic teams (building house)

Drawback: Needs constraints & collaboration

6. Big Bang Model

• Build everything quickly without much
planning - test later.

Best for: Small projects & Experiments

Drawback: Risk of failure.

Looking without
recipe sometime
dashy or
disaster

SDS → guide, provide complete documentation

- Acts as blueprint
- Avoids miscommunication
- Reduce costs and bugs

Characteristics

- Correct - accurate &
- Complete - include all features
- Verifiable - Tests are used to confirm
- Traceable - requirements traced from start to end
- Modifiable - update & change
- Consistent - no conflict
- Unambiguous - only one clear meaning

Components

Introduction - product purpose, scope, users, abbreviations

Overall description - product perspective, functions, characteristics

Functional Requirements - Action, operations

Non-functional Requirements - behaviour → performance
scalability, flexibility, reliability

External Interface Requirements → user, hardware, software

System Features → login, dashboard, payment communication

Constraints Legal, ethical, hardware limitations

Assumptions & Dependencies

↳ internet is always available ↳ external API, Third party service

Requirement Engineering

- collecting, analysing, documenting and managing requirements.
- ⇒ 5 major acts

1. Requirements elicitation (getting or gathering requirements)
2. Requirement Analysis (Analyzing, refining and prioritizing requirements)
3. Requirement Specification (Writing requirements in standard form)
4. Requirement Validation (Review, prototyping, test case generations)
5. Requirement Management (version control, change requests, maintain history)

Software Design Concepts

- How the software built internally.
- Acts like blueprint.

1. High Level Design (HLD) ⇒ Overall system architecture
 - * System architecture
 - * Database design
 - * Flow, ER diagram
 - * Modules & their relationships
 - * Technology stack
2. Low Level Design (LLD) ⇒ Internal design of each module
 - * Class, method, functions, Algorithms
 - * API, database table structure, workflow.
3. Abstraction ⇒ showing only essential details & hiding unnecessary complexity

4. Modularity $\Rightarrow$ Breaking the system into small modules easy to test, develop, maintain, update

5. Cohesion $\Rightarrow$ How strongly the elements of a module are related

• Validate user, password, credentials, Error return to dashboard $\Rightarrow$ High Cohesion = Good

6. Coupling $\Rightarrow$ One module depends on another module

• Payment module depends on order module $\Rightarrow$ Low coupling = Bad

7. Information hiding $\Rightarrow$ Hiding internal details

8. Functional Independence $\Rightarrow$ high cohesion, low coupling

9. Refinement $\Rightarrow$ Big task into smaller

10. Design Patterns $\Rightarrow$ Reusable solutions for common problems

• Singleton, factory, observer, model-view-controller.

Software testing

* Check the software work correctly or not

* It meets the requirement needs

Goal: • find defects

• Improve quality

- Manual Testing $\Rightarrow$ Check by human testers
- Automated Testing $\Rightarrow$ Using tools (selenium, JUnit, TestNG)
- White Box testing $\Rightarrow$ internal logic eg: loops, conditions, statements
Programmer/Developers
JUNIT, NUnit
- Black Box testing $\Rightarrow$ Functionality eg: Enter login details
Not known about internal code
Selenium, Appium
- Grey Box testing $\Rightarrow$ Partial knowledge of code.

Levels of Testing

1. Unit Testing [Test individual functions
Ex: add()
2. Integration Testing [interaction bt modules
Ex: checking login - dashboard
3. System Testing [Test the whole system
Ex: Apps
4. User Acceptance testing [works together
check the software meet business needs
Ex: check before final deployment

Function Testing $\rightarrow$ What the system does
ex: login, signup, payment

Non-functional Testing $\rightarrow$ How it behave or perform
ex: performance testing, load, stress, usability

Alpha & Beta Testing
 $\Downarrow$ Developers Organization
 $\Downarrow$ Customers and users

Verification
 $\Downarrow$ developers
 built correctly

Validation
 $\Downarrow$ end users
 meets their need

~~Smoke~~ Testing
 $\Rightarrow$ quick test to check the basic and critical features of software

cake is made correctly

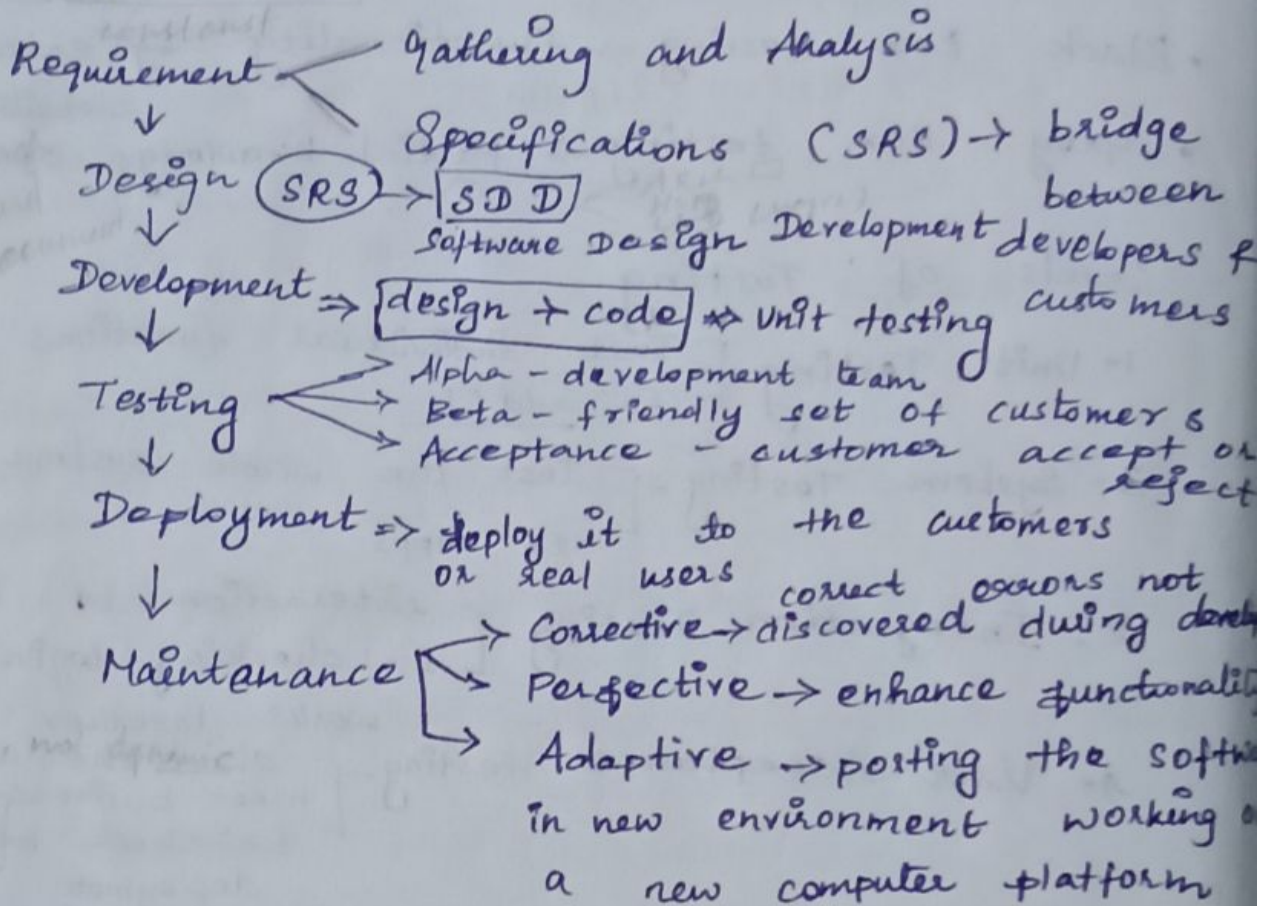
Customer wanted the cake or they like the cake

Waterfall Model

Traditional Software

⇒ Linear and Sequential

⇒ Requirements well defined & goals are clear



Features

1. Sequential Approach - each phase completed before moving to next phase
2. Document Driven - project well-defined, clear goal
3. Quality Control - emphasis on quality control
4. Rigorous Planning - scope, timeline, deliverables

Advantages

easy to understand
individual processing
properly defined, Documented
Working well for smaller projects

Disadvantages - No feedback

- No overlapping
- Limited flexibility
- Difficult to accommodate change requests
- Lengthy cycle
- Late defect detection

transforms
requirements into
implementable
specifications.

Design phase

↓
between requirement &
Software development

1. Architecture Design

- * Overall structure
- * Technology selection (requirements)
- * System → components, service and modules
- * Integration
- * Scalability [for future growth]

2. UI/UX Design (user-friendly / attractive) ↑ looks ↓ feels

- * User ~~des~~ Journey Mapping: (path)
Login → complaint lodge → Submit → Track
- * Wireframing & No color (Sketches)
↳ Where & how arranged.
- * Prototyping (movable) clickable & feedback
- * Visual Design → colourful
- * Accessibility elder, blind, visually impaired

Database Design

→ data will be stored, retrieved and managed

* Data Modeling → ERD (Entity Relationship Diagram)
table, relationship, cardinality

* Schema → structure, column, datatype, constraint

* Normalization → ^{remove duplicate data} reduce redundancy & improve integrity
↓
avoid repetition

* Indexing - optimal query performance ^{correct + consistency}

* Data migration planning - import existing data

Technical Specification creation

* Modules Specification functions

* API Design → endpoints, request/response

* Coding standard → code structures

* Interface Specification communicate

* Algorithm Design build logic

Risk Analysis and Mitigation

* Security Risk Assessment: vulnerabilities & plan security measures

Performance Risk Analysis:

Integration Risk evaluation challenge in third party analysis

Technology Risk Review

Mitigation Strategy Development:

Quality Verification and Validation

⇒ meets requirements

* Review:

* Validation:

* Traceability:

* Testing:

* Quality checks:

Windows OS Development using SDLC

Requirement Analysis | what are things that the system should do

- Fast booting
- Beautiful UI
- Strong security
- Support wifi, Bluetooth, printers, camera
- Multi-tasking, multi-users

Planning How to do, who will do and when

• kernel [Core]

• File system team NTFS, FAT

• Driver team keyboard, mouse, wifi

• UI/UX team start menu, taskbar

• Security team windows defender

• Networking team TCP/IP, wifi

System Design ^{make, design, (plan) to describe} ^{How will work}

1. High Level Design (What components should exist)

→ core component
kernel service of OS.
• core brain of windows ^{primary interface for hardware & software}

→ control process, memory, device, security, scheduling
sub components

object, I/O, process, memory, plug & play manager
languages: C, C++, Assembly

Tools: Microsoft Visual Studio, Windows Driver Kit, WinDbg (debugging),
CodeQL → security scanning

Device Drivers ^{hardware} (communicate with windows)

keyboard, mouse, Wi-Fi, USB, audio driver

Tools: WDK, HLK (Hardware Lab Kit)

Windows Subsystem (Win32 API)

Application to run

chrome, windows app like VS code, game uses

Win32 API
→ Create Window
→ Open File

language C, C++, C#

→ high-level framework

Tools: SDK, VS code

File System (Manage storage on disk)

• NTFS - secure, fast, journaling

language
C

• FAT32 - simple, old, used for USB drives

Tools
WDK
STT

• ExFAT - large USB/external drives

Features: encryption, permission, links, compressions

Windows Shell

(GUI) → for windows

- Taskbar
- Start menu
- File Explorer
- Desktop

Language: C++, WinVJ

Tools: VS, Blend, WinVJ

(SDK)

collection of tools, libraries, code.

without starting from scratch
debuggers, documentation, compilers, (API) diff.

application share data & communicate each other.

Security Layer

SAM → store passwords / credentials

UAC (user account control) → allow this app to make changes

ACL (who write/read)

Languages: C, C++

Tools: SDK, codeQL, fuzzing tools

Networking Stack

TCP/IP, DNS, firewall, Wi-Fi, Bluetooth, VPN

Language: C, C++

Tools: Network monitor tools, Wireshark, WDK

Boot Manager (BOOTMGR)

Boots up windows

Loads:

- Boot configuration data
- kernel
- Drivers

Language: Assembly, C

Tools: Windbg, Boot Configuration Data Editor

2. Low level design

process scheduler (runs on CPU)

tools: Windows Performance Analyzer, Event Tracing for Windows

Memory Manager

RAM, VM, paging, address management

tools: Resource monitor, Performance monitor

I/O Manager

File, Network, Device I/O

tools: I/O Meter, Disk Test Tool, WDK

Graphics Subsystem

Render, gaming

language: C++, shader HLSL

Registry Structure

contain hardware info, installed programs, user setting, device setting, OS configurations

HAL

hardware abstraction layer

Windows run on different hardware without rewriting kernel.

CPU difference, controllers, memory mapping

tools: WDK, ACPI

Implementation / coding

Languages

C → kernel

C++ → UI components

C# → APPS & tools

Assembly → Boot process

Testing

1. Kernel testing → memory leaks, kernel crashes
2. Driver testing → printer, graphics card, USB
3. Security testing → attacks, Ransomware
4. Performance → Boot time; battery life, memory
5. Compatibility → old apps should still run

Deployment

- ISO image (single file contain many files)
↳ maintain structure
- Gives to manufacturers (Dell, HP, Lenovo)
RTM Release to Manufacture
Maintenance final version
- Security patches, Bug fixes, driver updates
UI improvements, feature upgrades.

RTM → finished, stable, tested,

Microsoft → company

OEM → Original Equipment Installation

add their drivers, branding

↓
Wifi,
touchpad,
camera, fingerprint

↓
Dell support,
hp support

UML diagram

- Visual language
- similar to blueprint in architecture

Need • Diagram for modelling software

Structural UML

Static how system is organised

- what is present
- show relationship, hierarchy, attributes
house
wall, room, door, window

Behavioural UML

dynamic aspects of a system

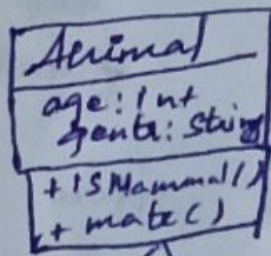
- focus on responds (behaves)

Open door
Enter kitchen
Switch on stove
prepare food

Structural

Class diagrams

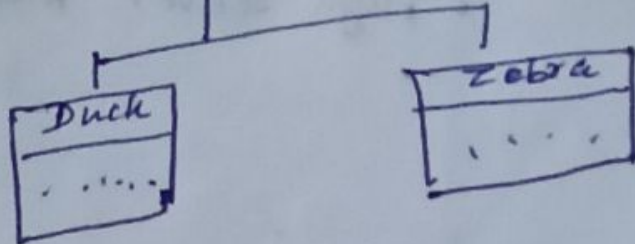
- Object oriented software system
- show class, method, attributes, relationship
car



color } → Attributes
speed }

start() } → Methods
stop() }

skeleton
house
blue print
design
structure

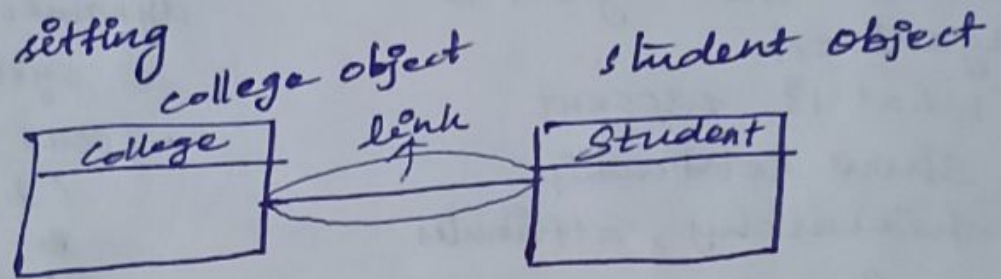


2. Object diagram

↳ instance of class (similar to class)
→ relationship between them at a point

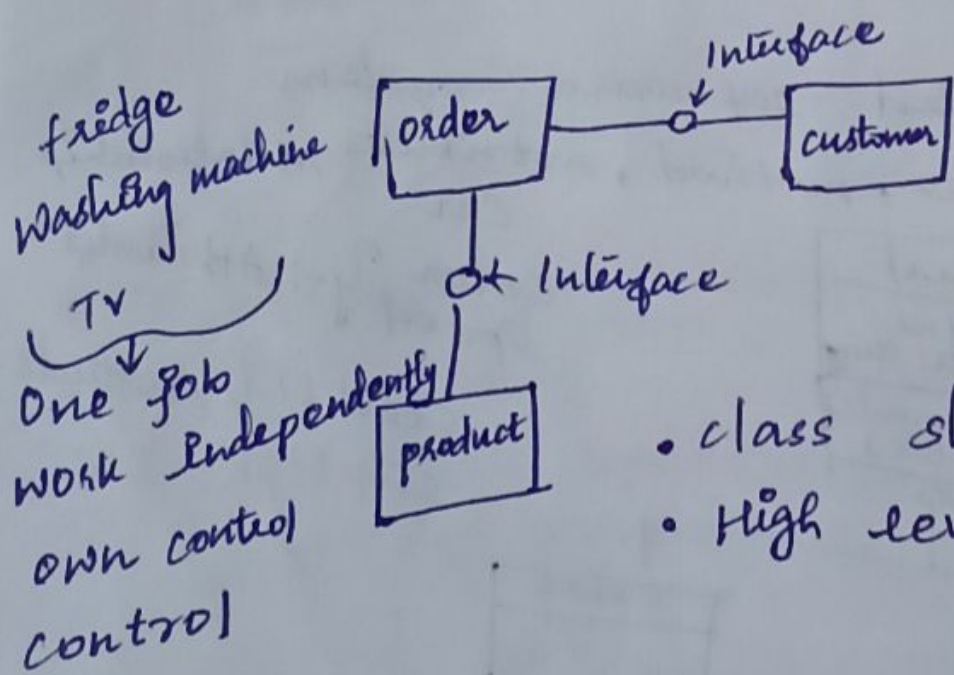
Class only design, object show real value
how looks actual
 show link

right know how
where furniture placed,
people sitting



3. Component diagram

• Interface are used to communicate with each other.



• Software elements help us in understanding functional requirements

• class show internal design
• High level modular structure

4. Deployment diagram (software + hardware mapping)

• System hardware & software representation.

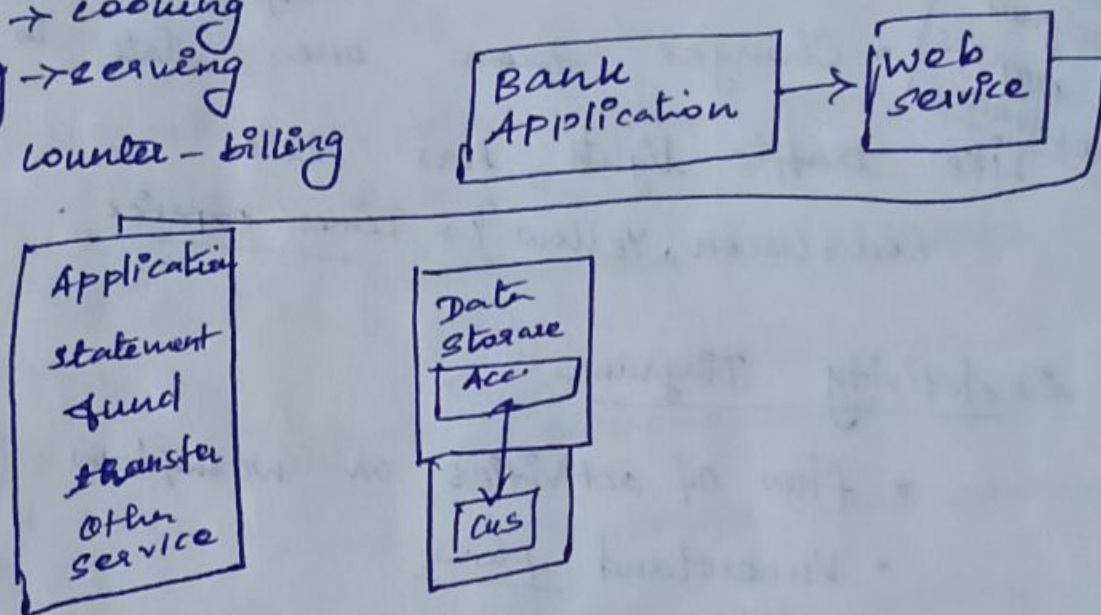
• Software deployed over multiple machines.

Hotel - Software • Where do parts run

Kitchen → cooking

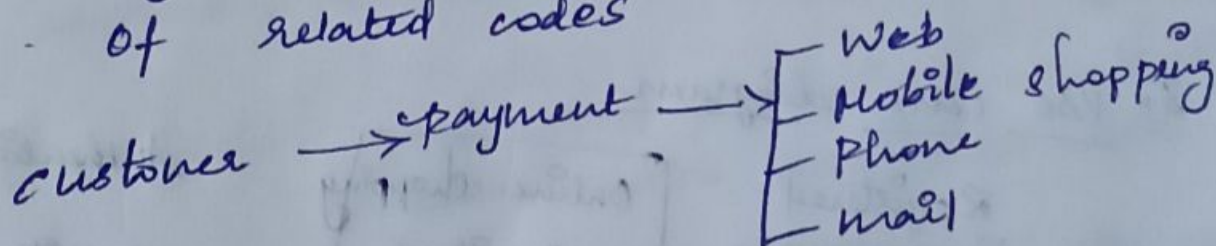
Dinning → serving

Billing counter - billing



5. Package diagram

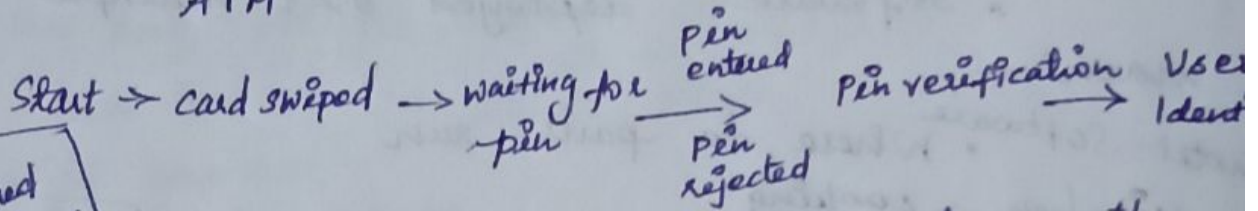
↳ organising and grouping of related codes



Behavioural UML diagrams (dynamic aspects)

1. State Machine Diagram

ATM



New Assigned
In progress
Resolved

• Changes from one state to another
Like traffic lights has states

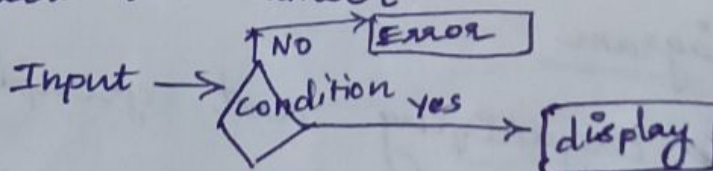
Red, Green, Yellow } → Timer expires

2. Activity Diagram

• flow of activities or workflow

• Understand flow

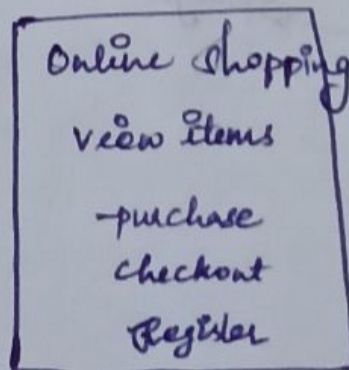
Check odd number



start
~~wakeup~~
user login
Complaint details
Submit Database
stop

3. Use Case diagram

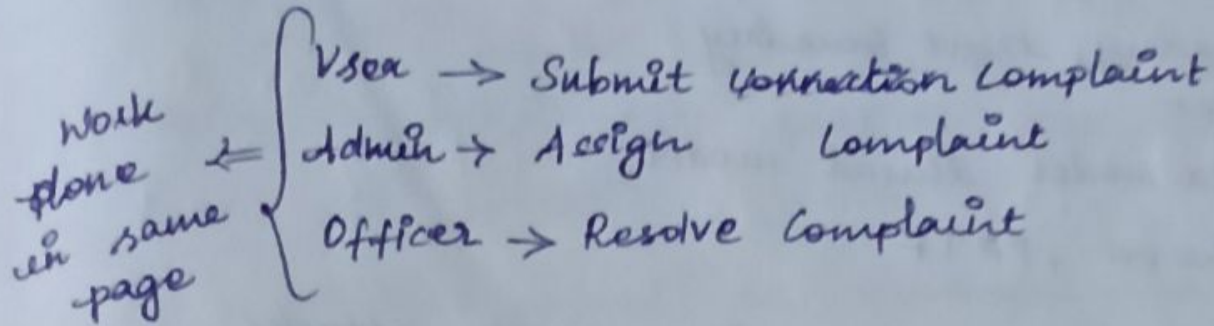
people → $\begin{matrix} \text{Registered customer} \\ \text{New customer} \end{matrix}$



Authentication
Identity provider
Payment service
Paypal

• Show functional requirement & how they interact with agents (actors)

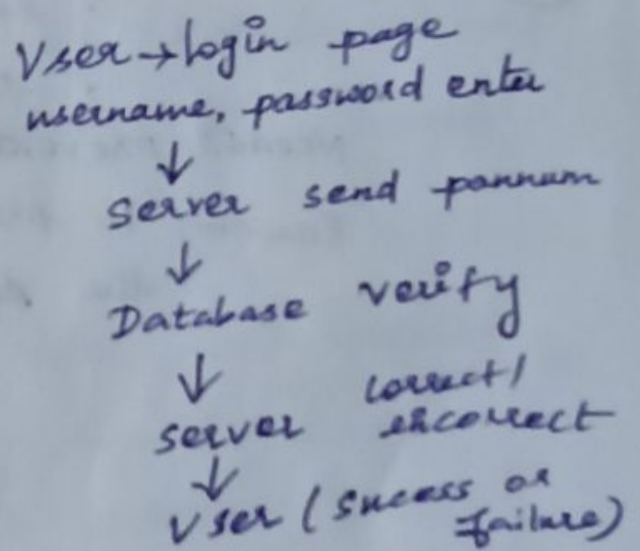
- High level view without going to the implementation details.



Sequence Diagram

- How and in what order the objects in a system function
- Used by businessmen & software developers

system events $\rightarrow$ execute in order



Connection Diagram

relationship oriented

Tools: Lucidchart: • web-based, friendly & collaborative
allow multiple users to work

Draw.io: • free, web-based, cloud storage services

Visual paradigm: full SDK, UML, ER, flowchart. professional software
 $\rightarrow$ install needs

StarUML: open source, UML tool, simple, new

Thread, weakness, vulnerabilities

Boundary, trust boundary

asset

data model thread model

Zoho PVR, PPTF

thread $\rightarrow$ harmful real time $\rightarrow$ disaster, attacks, breakdowns

threadmodel $\rightarrow$ • assert (valuable things) what go wrong
• who can attack (threads)

harmful $\rightarrow$ harm

• Vulnerabilities (weakness)

• entry points (who can enter)

• protection / controls / Mitigations

Needs: prevents problem before they happen

Reason: To protect asset, weaknesses, reduce the damage, money loss, built trust,

Thread Model

Thread

=> It means that what are the things, actions that creates the fear or make us afraid.

=> It should be harmful or cause some damages for us.

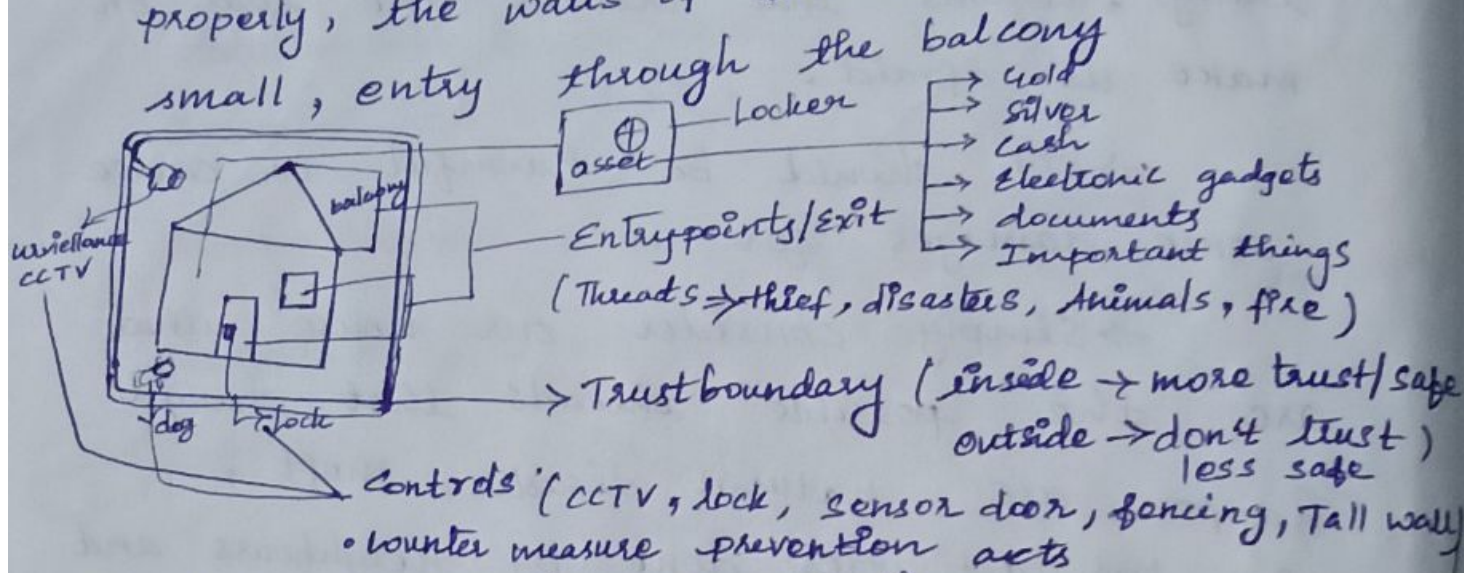
=> Simply consider our house what are the possible threads that should happen are natural disaster, theft, harmful activities done by neighbours and the childrens playing near the house

Thread model

=> Helps to consider all possible ways for the thread should ^{what should go} happen, it provide the way to prevent that before it should happen.

- In our house we need to protect the valuable things like gold, silver, important documents, valuable gadgets these are valuable assets that should be protected by using lockers.
- asset -> valuable thing that should be protected
- Enhance the security by MFA, authentication

⇒ It should consider the weak points like what are things that cause damage to our house like house was not locked properly, the walls of the house was too small, entry through the balcony



Trust boundary (based on trust level)

⇒ Level of trust changes

⇒ Every one should have the trust level, according to the trust level their boundary should be limited.

family member → high Trust level
known relatives ↓
Neighbours
unknown / strangers

Data Model

* Structure of data / how they arranged & what are the attributes they should have

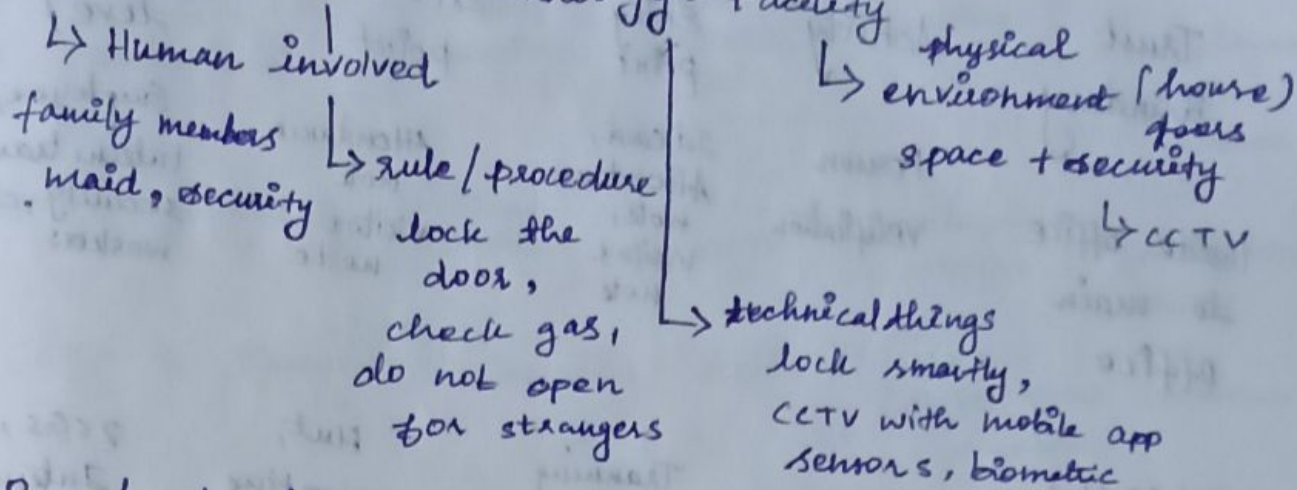
House ⇒ Family members, rooms, asset, security, visitors

Data Flow

* How the data flow one place to another

Visitor → Bell → Family $\xrightarrow[\text{reject stay outside}]{\text{allow enter inside}}$ decision

People - Process - Technology - Facility



* People → who use & test & Manage systems

* Process → steps, rules or procedure
policy, plan, backup, access

* Technology → Tools Antivirus, firewall, MFA, Encryption
What? → to protect data

* Facility → where → environment - security + infrastructure

Thread Model architecture

Trust boundary → a line where the trust changes

Entry Endpoints → where something enters
login, endpoint, upload, payment point

Exit point → where the data leaves
server to users, download, email sent

Trust level given → how much trust should be to something (guest user & low trust)

Activity → what each one can do
(view, write, edit, admin actions)

Trust Boundary	Activity	Entry point	Exit point	Trust level
Inside office to main office	Person Verification	Id card, Attendance note, Visitor note	Attendance note, Visitor note	Employee, Intern, trainee, security, chef, workers
Hut (inside) Zoho guest Zoho hut Treehouse Mandarscha	ZSGS training	Training session, Zoho guest, ZSGS account	Hut, meeting and	ZSGS, Intern
Main gate to hut Visitor meeting Interview conduction Outside office to Main office	Access Verification	Main/back gate resume, address, call ID card	Main/back gate check out	Visitor Employee trainee security Intern

Inside Office	SOC (monitoring)	Z-pivi (wifi)	Browser	
	Thadani development	URL Browser	DNS resolver controller	
	- Meeting	IP address	application	Employee
	- Repository access	Terminal gpt pro	Git repo, Build dashboard	
	- AI use	VSCode,		
	- node development	eclipse, mysql, package	local machine testing	

Meeting host
to participants

Attend
meeting

Host
Network,
url.

Recordings

Employee
ZSAS
Trainee

CCTV to monitoring

Live
capturing

CCTV

Monitoring
device

Anonymous
user

CCTV to Recording

Recording

CCTV

DVR
(digital video
recorder)

admin

Threat modelling

→ attacks, threat
* identify, communicate and
understand threat

* Representation of the all information that
affects the security of an application

* Also consider the threads that should
be challenged in future.

include

→ Deception

→ Assumption that can be checked or
challenged in future

→ Actions that should be taken to
mitigate or control each threat

Objectives

→ Improve security by identify threads

→ define countermeasure to control
↓
action, control or safeguard to
reduce or eliminate threat

→ May be malicious (such as DOS, ^{make the system too busy}
attack) or / Incidentel / failure of (SD)

intentionally
harm

fake, steals

make the
system unavailable for
normal users

Malicious Software

* program or code created to harm our system (no computer) or steal info.

◦ malware → steal

◦ Ransomware → Break

◦ Worm spreading → spread to neighbours

◦ Virus → infects your files & spread when started

Malicious User

* Intentionally exploits weakness in a system.

⇒ harm or steal data

Sql injection (SQLi)

* Type of security attack

* Malicious sql code into web application
↳ intentionally access, modify, delete data

Cookies

⇒ Small piece of data that a website stores on computer

⇒ It store & knows who you are and your preference.

Keep the track of the user preference

Thread Model Across the life cycle

* Not one time task.

* Update threat models is advisable after events

- A new features is released
- Security incident occurs
- Infrastructure change

Four Question Framework

1. What are we working on? (What you are protecting)
2. What can go wrong? (Identify threads or vulnerabilities)
3. What are we going to do about it?
4. Did we do a good job? (Countermeasures)
- (Actually works the security measure)

Thread Modelling Process

* Identify, quantify and address the security risks

* Decomposed into four high level steps

1. Scope your work

[components interact]

⇒ Drawing diagram, data flow diagram

⇒ Identifying entry points

⇒ Identify asset

how the data moves
where people can enter and exit

Determine Threads

2. Identify Entry Points

=> Where Interact with the application

Entry points = doors, windows, garage

Anywhere someone can enter

Software entry points = login page, APIs, form

determine countermeasure ^{inputs} - upload buttons
+ mitigations

3. Identifying Assets

=> Protect important data, functions, features

assets = user data, passwords, payment details,
business logic

Identify trust levels

=> how much access different users
or system have.

admin, normal users, guest users, external systems

Reading or Creating User Stories

How the user will use the app.

countermeasure (controls main exist should select one)

↳ selection include factors like
attack, damage, complexity, cost

Mitigation strategy

- Accept → small crack in wall, cost is high, accept & leave it
- Eliminate → if the door is broken don't lock it, total alarm
- Mitigate → front door is okay but not strong, add lock, camera, install alarm
- Transfer we have risk, but you won't pay it
if we have house insurance, company should pay for any theft

Step 1: Assess your work

* How the application should interact with external entities.

* Information gathering & documentation, ensure that correct information is collected.

Thread Model Information

Credentials

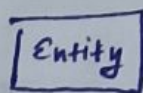
- Application Name -
- Application Version
- Description
- Document owner
- Participants
- Reviewer

↳ user name + passwords

↳ Anything that helps the system confirm your identity

Data Flow Diagram

→ data comes from, where it goes, how it is processed, where it is stored.



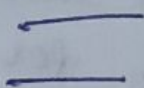
External Entity



Process



Multiple process



Data Store (data stored are not modify / only store)

→ data flow within the application
↳ denotes direction

----- Trust boundary ⇒ represent the change of trust levels
⇒ show the location where the level of trust changes.

Determine threads (STRIDE) ⇒ identify threads

* find what can go wrong

Spoofing - try to access another user credentials

↳ Authentication (stolen passwords, fake login, fake authentication) → confirm user, who are you?

Tampering - modifies data (manipulate data)

↳ changing values in database, modify files

Repudiation - Performing prohibited operation lacks trace. (escape responsibility)

Information disclosure (leaking information)

↳ Private/secret information get exposed

Denial of service (too busy)

↳ Attack overloads, make the web server unavailable.

Elevation of Privilege (Attackers become admin)

• Authorization (what permission do you have?) give user permission to access resource

Phishing Attack

* Cyber attacks

* Attacks act as a trusted person to steal the sensitive information

Social Engineering

- Cheating the human's mind
- Not hacking the system
- Hacking people's curiosity, fear, people, Trust

↓
password, banking detail, session tokens, visitor pass, digital ID or proof of identity
a system gives you when you log in.

* Phishing is the online fraud

↳ To get your private information

passwords, credit cards, bank account data

attacks that comes from fake trusted sources.

Attackers mimic the website or webpage of the trusted secure platform.

Happens while doing

- Clicking on unknown file
- Using an open or free open wifi hotspot
using wifi, owner should controls the user's data.
- Responding to social media request
- clicking unauthenticated links or ads.

Types

• Email phishing:

* clicking unverified spam emails and leaking secret data.

* Use the information to steal money from user's account.

• Spear Phishing:

* target particular user, sends malicious mails to that particular person.

pretends like

Ex: The manager of that employee to transfers a large sum of money. (recently the payment details using given link)

• Whaling

* Same as spear phishing

* Main target is the head of the company. (ex). Financial information for submitting false tax returns & claim refunds. (Tax scams)

• Smishing

* Same as e-mail

* Using the SMS

• Vishing

* Voice phishing (ex: credit card numbers or confidential data from user)

Clone phishing

* The attackers copies the email messages that were sent from a trust source.

* Adding a link that redirects the victim.

Causes

Financial Loss → credit card no.s

Identity Theft → steal information

Damage to Reputation client may lose trust

Disruption to Business Operations: lost production of data

Spread of Malware infects victim computers

Signs of Phishing

- Suspicious email addressing - fake email address
- Urgent requests for personal information -
- Poor grammar and spelling
- Request for sensitive information
- Strange URL
- Unusual link or attachments

How to ^{stay} protected?

- Authorized Source
- Confidentially
- Check URL
- keep the firewall on
- Avoid replying to suspicious thing
- Phishing Detection tools
- Try to avoid free wifi
- keep your system updated

How to Distinguish Between a Fake Websites and a Real Websites?

- check the url of the website
https-~~x~~secure (encryption to transfer data)
- Check the domain name of the website
Amazon x amazon
- Look for site design
www.8garcabe.com/Facebook, www.facebook.com
- Check the availability web pages:
only display login page

Anti-Phishing tools (to detect phishing)

- APDA (Anti Phishing Domain Advisor)
- PhishTank (verify reports of phishing attacks)
- Webroot Anti-Phishing (ml algo to identify & block phishing websites)
- Malwarebytes Anti-phishing (detecting & blocking suspicious websites)
- Kaspersky Anti Phishing [browser extension
real-time protection, db + security tools]
→ (watching everything inside the computer)

Antivirus

protect the system or mobile device
from harmful programs

SSDLC

Secure SDLC

Requirements

What data should be protected?

Who can access what?

What rules should we follow?

Design

Define trust bounds

Choose secure design pattern

Identify threat in thread modelling

Development

Developers follow secure coding

Avoid vulnerabilities

SQL Injection

Broken authentication

Cross-site scripting

Use code reviews and static code analysis

Testing

penetration testing

Vulnerability

Fuzz

Security testing

Maintenance

Patch vulnerabilities

Monitor logs

Update dependency

Incident response

Deployment

HTTPS enforcement

Secure server configuration

Secret management

Access control setup

Needed

Why SDLC is needed?

fewer vulnerabilities

Safer application

Trust from users

Compliance with standard

Example
Amazon

User password

Payment details

Order history

User address

Product inventory

} Acc

Attackers

Hackers

Fake users

Inside employee

Competitors

Design

Entry points

login, signup,

add to cart

Search

payment gateway

Admin

Trust boundary

user's boundary } → server

untrust → trust

Server → Payment
Gateway

trust → external
dev

user → admin
different level

Design Sec Security Conts:

Rate Limiting, Captcha, Hashing passwords,

input validation, MFA

} Login

prepared statements, input statements, output

encoding

} search bar

Token based authentication,

validate product price on server,

log unusual activity

} → cart API

Enforce HTTPS, encrypt payment data,

Verify transaction with payment,

secure API keys

} → Payment

Stop SQL Injection,

stop price manipulation

encrypt sensitive data

} → development phase

Types of Cyber Security

Network Security:

Protect ~~from~~ unauthorized
access, data > theft
House / Building

Security guard → Firewall
CCTV → Intrusion, Detection
System.

private access to family members } VPN

Separated by wall or doorway } Network Segmentation

Pest control → antivirus,
security

firewall → who can enter or leave.
known → enter, unknown → block

IDS Intrusion Detection System

CCTV → detects suspicious things
alerts does not block

VPN → private tunnel between
you and Internet

Nobody can see where are you
going, what are you carrying

→ Protect Internal network from
outside threat.

Computer Network
Share information,
file, resource and
communicate.

School
(principal, students,
Teachers)

all are connected
using the cables
or wifi
share notes, ^{access} server,
print documents,
same internet

Types of Computer network

- LAN (Local Area Network)
Small area school, home
- MAN (Metropolitan Area Network)
Medium area - city level network
- WAN (Wide Area Network)
Large area like connecting countries

Connected by
Cables, wifi, Bluetooth
Optical fiber,
Satellite

Devices

Router, switch,
Modem, access
points, ...

Components

Router → connects the device by internet

Switch → connects many computer inside LAN

Modem → brings internet from ISP → internet service provider
Access point → provide Wi-Fi

Server → store data & apps.

1. Modem → internet from ISP to office/house
2. Router → decides the internet where to go
↓
Security gate who should go, how to route, keep outside away without permission

3. Switch → connects many computers inside the local Area Network (LAN).

Ex: Building

Big lobby has multiple doors
people the lobby move from one room to another room without going outside (internal connection)

4. Access point (AP) Wi-Fi (connect devices using radio waves without physical cable)
provide Wi-Fi
• Broadcast signals.
• connect/listen wirelessly
(speaker → announce msg without connection)

5. Server (store apps, websites)

store data

manage access

Hosting websites

Client → One who taking things

Server → One who offering things

SSDL → Secure SDLS
↳ ensure that the programs are built with security in right mind

Key principle:

- Security by Design integrate security in SD planning & requirements & Design
- Continuous Monitoring → security checks for check every step ongoing issue.
- Risk Assessment identify, prioritize, mitigate
- Education & Training continuous training to solve the issue effectively
- Teamwork and collaboration → team work for common security goals.

SDLC process

Requirements

Design

Development

Testing

Deployment

SSDLC

Risk Assessment

Threat Modeling

& Design review

Static Analysis

Security Testing &
code review

Security Assessment &

Secure Configuration